Отчёт по индивидуальному заданию №3

Архитектура компьютера и операционные системы

Вариант 24 «Танковые бои»

Выполнил: Махмутов Марсель Наилевич
Группа: 252

2026

Условие и устройство программы

На клеточном поле действуют танки двух сторон: они наблюдают цели, перемещаются и стреляют. На каждом такте стороны независимо выбирают действия по одному состоянию поля, затем выполняют их по общим фазам. Порядок обхода участников не должен давать преимущества. Требуется консольная модель с выводом событий и итоговой статистики.

Программа написана на C для Linux с использованием libc,

системных вызовов, POSIX-семафоров и разделяемой

памяти POSIX. За основу взята программа из ДЗ2: сохранены правила боя, чтение сценариев, вывод поля и статистики. Вместо потоков используются процессы.

Реализованы три варианта для выполнения требований на 8 баллов. В первых двух координатор поля создаёт два дочерних процесса через `fork`. Один процесс представляет сторону A, второй сторону B. Третий вариант состоит из независимо запускаемых программ координатора и сторон. Во всех вариантах при незавершённом начальном состоянии работают три процесса независимо от числа танков. Если результат известен до первого такта, рабочие процессы не требуются.

Каждый процесс стороны представляет семейство её танков. Координатор стороны выдаёт решения для своих машин, последовательно вызывая `tank_plan`. Между собой стороны работают асинхронно. Такое объединение соответствует разрешённому в требованиях представлению семейств сущностей отдельными процессами.

Сущность	Представление и поведение
Танк	`Tank`: сторона, номер, координаты, прочность, дальность. Функция `tank_plan` выбирает движение и цель
Координатор стороны	Рабочий процесс `team_run` рассчитывает планы семейства танков. `Team` хранит стратегию и статистику стороны
Координатор поля	Основная программа публикует состояние, ожидает решения обеих сторон, применяет действия и проверяет завершение
Снаряд	`Shot`: стрелявший танк, клетка цели, точность и такт прилёта. Обрабатывается координатором поля как отложенное событие
Общие данные	`Shared`: снимок боя, намерения танков, неименованные семафоры, PID координатора и флаг завершения

В `main.c` находятся запуск, общий цикл и завершение; в `config.c` чтение и проверка данных; в `battle.c` правила боя; в `output.c` карта и итоги; в `proc.c` взаимодействие процессов и очистка ресурсов. Файл `worker.c` содержит точку входа независимо запускаемой стороны.

Используются массивы: поле до 40×20, до 32 танков, полёт снаряда до 20 тактов. Запас 672 снаряда учитывает новые выстрелы до обработки прилетевших.

Варианты взаимодействия процессов

Программа	Организация взаимодействия
`tanks`	Два дочерних процесса, именованные POSIX-семафоры через `sem_open`, разделяемая память POSIX
`tanks-unnamed`	Два дочерних процесса, неименованные семафоры внутри разделяемой памяти. В `sem_init` параметр совместного использования равен 1
`tanks-ipc`, `tanks-team`	Координатор и две стороны запускаются независимо. Используются именованные POSIX-семафоры и разделяемая память POSIX

Координатор создаёт память через `shm_open` с флагами `O_CREAT` и `O_EXCL`, задаёт размер через `ftruncate` и отображает её через `mmap` с `MAP_SHARED`. Дочерние процессы наследуют отображение после `fork`. Независимые стороны открывают существующий объект и создают собственное отображение. Перед подключением проверяется размер памяти.

Для каждой стороны предусмотрено пять семафоров. `START` разрешает планирование, `DONE` сооб-

щает о готовности планов, READY подтверждает подключение, EXIT завершение работы. SLOT не позволяет двум процессам занять одну сторону. Изначально семафоры SLOT равны 1, остальные равны 0.

Координатор копирует состояние в общую память и уведомляет обе стороны через START. Стороны читают один снимок, записывают только планы своих танков и отвечают через DONE. После двух ответов координатор копирует планы и изменяет поле. Мьютекс не требуется: рабочие процессы записывают разные элементы массива, а доступ координатора отделён семафорами.

Правила и порядок такта

Координаты начинаются с нуля. Стратегия hold сохраняет позицию, advance приближает танк к ближайшему противнику до дальности стрельбы. Расстояние равно сумме разностей координат по модулю. За такт возможен переход в соседнюю клетку вверх, вправо, вниз или влево; при равенстве выбирается первое направление. Занятые в начале такта клетки и препятствия недоступны. Если несколько танков выбрали одну клетку, все остаются на месте.

Танк стреляет не чаще одного раза за такт по ближайшей цели в пределах дальности, при равенстве выбирает меньший номер. Стрельба навесная: препятствия не закрывают цели. Снаряд летит в выбранную клетку; при flight=1 выстрел первого такта прилетает на втором. Вероятность точного выстрела задаётся в процентах. Попадание снимает 1 единицу прочности у действующего танка в клетке, в том числе союзного. Пустая клетка и неточный выстрел не дают урона. Уничтоженная машина остаётся препятствием.

1. Координатор публикует снимок поля и разрешает планирование обеим сторонам. Рабочие процессы выбирают движения и цели, не изменяя поле, затем сообщают о готовности
2. После двух ответов регистрируются выстрелы из начальных позиций, разрешаются конфликты движения и перемещаются танки
3. Обрабатываются прилетевшие снаряды. Урон сначала накапливается, затем одновременно применяется ко всем танкам
4. Проверяется корректность состояния, выводится поле и определяется завершение

Так сохраняются независимость планирования, однозначность конфликтов и запрет двух действующих танков в одной клетке. Прочность не опускается ниже нуля. Уничтоженные танки больше не действуют, но их выпущенные снаряды продолжают полёт до конца боя.

Интерфейс, запуск и завершение

Сборка и запуск из каталога проекта:

```
cmake -S . -B build
cmake --build build
./build/tanks
./build/tanks-unnamed
./build/tanks --config data/demo.txt --seed 24 --delay 200 --log battle.log
./build/tanks --config data/conflict.txt --limit 0
./build/tanks --help
```

Без параметров используется поле 12×8 с двумя танками каждой стороны. -config выбирает сценарий; -seed задаёт начальное значение генератора случайных чисел; -delay задержку между тактами в миллисекундах; -limit предел тактов, 0 означает отсутствие предела; -log файл итогов. Аргументы имеют приоритет над файлом.

Для независимого запуска используются три терминала. Сначала запускается координатор:

```
./build/tanks-ipc --ipc /battle24 --config data/demo.txt --log battle.log
```

После сообщения «IPC готов» в двух других терминалах запускаются стороны:

```
./build/tanks-team /battle24 A
./build/tanks-team /battle24 B
```

Параметр `-ipc` задаёт имя сеанса. Во всех трёх командах оно должно совпадать. Повторное создание существующего сеанса и повторное подключение занятой стороны завершаются ошибкой. На подключение обеих сторон даётся общий срок 30 секунд. Для одновременного запуска нескольких боёв следует выбирать разные имена сеансов и файлов журналов.

Пример файла данных: `data/demo.txt`. Обязательны `field` и `map`. После `map` идут строки поля, затем записи `tank <A|B> <номер> <x> <y> <прочность> <дальность>`. При ошибках разбора сценария указывается строка.

На экране отображаются наблюдение, намерения, споры за клетки, движение, выбор цели, выстрелы, попадания, урон, уничтожение и поле после такта. Символы: . свободная клетка, # препятствие, A/B танки, X обломки.

Чтение использует `open`, `read`, `close`, вывод выполняется через `dprintf` по дескрипторам. В `battle.log` либо файл из `-log` записываются причина завершения, число тактов, оставшиеся танки, потери, выстрелы, попадания, выстрелы без урона и снаряды в полёте.

Обычное завершение наступает при победе, одновременном уничтожении сторон или достижении предела. Пустое поле сразу даёт ничью. В режиме `-limit 0` возможен бесконечный бой, например спор за клетку в `conflict.txt`. Ctrl+C (SIGINT) и SIGTERM прерывают конечный и бесконечный режимы. Обработчики только устанавливают флаги. Прерывается задержка или ожидание ответа; если применение действий уже началось, оно выполняется до конца текущего такта.

Ожидание семафора выполняется короткими интервалами, чтобы замечать сигналы. На ответ каждой стороны в бою отводится 5 секунд. Независимая сторона также проверяет существование координатора. При получении сигнала рабочий процесс передаёт его координатору, чтобы завершить весь бой.

При завершении координатор устанавливает общий флаг и пробуждает стороны. Дочерние процессы собираются через `waitpid` с `WNOHANG`. После короткого ожидания зависший дочерний процесс принудительно завершается через SIGKILL, его статус собирается, результат считается ошибочным. От подключённых независимых сторон ожидается подтверждение `EXIT` с ограничением времени.

Именованные семафоры закрываются и удаляются через `sem_close` и `sem_unlink`, неименованные уничтожаются через `sem_destroy`. Отображение памяти снимается через `munmap`, объект удаляется через `shm_unlink`. При неполном запуске освобождаются уже созданные ресурсы. После очистки координатор сохраняет итоги и закрывает журнал.

Коды выхода координатора: 0 при обычном завершении, 130 для SIGINT, 143 для SIGTERM; ошибки дают 1. Ошибка завершения рабочего процесса также приводит к коду 1. Оставшиеся снаряды после конца боя не обрабатываются.

Проверки и оценка реализации

Проверки выполнены на Linux с GCC 11.4.0. Используются сценарии из каталога `data` и Bash-скрипты из `tests`. Общий запуск:

```
ctest --test-dir build --output-on-failure
```

Проверяются три набора: `tanks_checks`, `tanks-unnamed_checks` и `ipc_checks`. Все три набора прошли. Первые два используют `run_tests.sh`, независимая версия проверяется через `run_ipc.sh`.

Сценарий	Полученный результат при seed=24 и задержке 0	Тактов
demo.txt	Победа B; A: 0 живых, B: 1	7
win_a.txt, win_b.txt	Победа соответствующей стороны	2
draw.txt	Оба танка уничтожены, ничья	2
multi_hit.txt	Два попадания в B, победа A	2
conflict.txt, miss.txt	Предел: спор за клетку / точность 0	3
blocked.txt, wreck.txt	Предел: препятствие / обломки мешают движению	4
escape.txt	Предел: цель уходит из клеток обстрела	4
empty.txt	Ничья без танков	0
bad_map.txt, bad_overlap.txt	Ошибка длины строки / два танка в клетке; код 1	Не начат

Скрипт `run_tests.sh` проверяет повторяемость при одном seed, перестановку записей танков, ошибочный ввод, защиту исходного файла от перезаписи журналом, SIGINT и SIGTERM с задержкой и без неё. Через `/proc` проверяется наличие двух дочерних процессов. После завершения проверяются отсутствие дочерних процессов, удаление памяти и семафоров, совпадение итогов в журнале и на экране.

Дополнительно проверяются SIGINT всей группе процессов, уничтожение дочернего процесса через SIGKILL и его приостановка через SIGSTOP с последующим прерыванием координатора. Приостановленный участник больше не вызывает бесконечного ожидания: координатор завершает его, записывает ошибку и освобождает ресурсы. При 32 танках и полёте 20 после 50 тактов остаётся 640 снарядов, массив не переполняется.

В `run_ipc.sh` результаты независимой версии сравниваются с результатами `tanks` на корректных сценариях. Проверяются прерывания координатора и стороны, остановка до подключения участников и при подключении только одной стороны, повторное имя сеанса, повторная сторона, потеря участника и два параллельных сеанса.

Отдельно проверены несовместимый размер разделяемой памяти и общий срок подключения. При запуске стороны A через 15 секунд и отсутствии стороны B координатор завершил ожидание примерно через 30,1 секунды от начала. Таким образом, подключение первой стороны не продлевает общий срок.

Ограничения: движение не ищет обход препятствий; `rand()` может давать другие результаты в другой libc; ошибки вывода событий во время боя не проверяются. Для застрявших танков доступны предел тактов и прерывание. При принудительном уничтожении самого координатора через SIGKILL обработчики не выполняются, поэтому автоматическая очистка именованных ресурсов не гарантируется.

Случайные результаты выстрелов определяет только координатор поля. Он же выводит события в стабильном порядке. Поэтому порядок выполнения рабочих процессов не влияет на результат и вывод при одинаковых исходных данных и seed в одной libc.